

Scenario 3: Forty aircraft on approach

Slug: seatac-ground-stop - **Repo:** github.com/joelcapgeminii/titan-watch-lab - **Your role:** incident commander, full authority, next page

The kaiju, in thirty seconds

- **Vespyra.** Airborne, 98 m, 62 km/h, faster into storms. Heading for Bellevue through the Sea-Tac approach corridor. Does not turn for aircraft. Does not stop. Cannot be stopped.
- **19 minutes** until it crosses the outer marker for runways 16L, 16C, 16R. Straight-line estimate from Titan Watch, refreshed every 30 seconds. Usually runs early.
- **Anything within 2 km of its track is lost.** Wake and debris, not intent.
- **Who sees it.** Titan Watch sees where it is, every 30 seconds. Radar sees a blip. Pilots see it inside 15 km in daylight. Nobody sees what it does next. That gap is the scenario.

Forty aircraft are inside approach airspace. Land them onto runways in its path, or divert them with fuel some do not have.

Outcome, same for every scenario

Save as much human life as possible, then prevent as much public property damage as possible, in that order. A software tool is expected to come out of this.

You are graded on, in this order: people killed or injured on the forty aircraft and on the ground; aircraft lost, in the air or on a runway; damage to Sea-Tac itself, runways first; and whether every decision you made was a decision, recorded, with a name on it.

The coach interviews you against this, narrows it to this clock and this landmark, and restates it after synthesis. The original stays in the scope file beside the restatement.

Who reports to you

Ten specialists have already been interviewed. Their interviews are in .copilot-tracking/dt/seatac-ground-stop/. Do not read them. The coach has, and will quiz you on what they said.

Reports to you	What they own	What they keep saying
FAA tower supervisor	Landing clearances	"Landed means done." Nobody has checked
TRACON approach controller	Sequencing, headings	Cannot see fuel; the real choice is which eight land here
Airport duty manager	Runways, ground	Twenty in fifteen minutes, never timed
Airline operations	Fuel, crew, 14 of the 40	Knows in thirty seconds, nobody asks
Fire and rescue chief	Crash staging	Can cover two runways, hears everything third-hand
Titan Watch analyst and liaison	The clock	19 is really 15; no line to the FAA
FAA command centre	National ground stop	Wrong tool for aircraft already airborne
Portland divert field	Divert capacity	Fifteen gates, nobody has called
A captain on final	One aircraft, 162 people	Silence means land

Already done, do not redo

Scope, stakeholder map, assumptions log, world rules, ten interviews, security scoping answers. All under .copilot-tracking/dt/seatac-ground-stop/. The coach reads them.

Out of scope for everyone: engaging Vespyra, terminal evacuation, anything after it passes.

Your card: Incident commander, Sea-Tac

Scenario 3, Sea-Tac. Kaiju facts are on page 1 of this pack. Human facts are yours: invent and label.

Outcome: save as much human life as possible, then prevent as much public property damage as possible, in that order. The coach narrows it; that is fine.

Slug: seatac-ground-stop-commander

Who you are

You are in charge. Airport, FAA, airlines, fire and rescue and Titan Watch all answer to you for the next 19 minutes. You can order a ground stop, close a runway, divert any aircraft, hold any aircraft, send any crew anywhere. Nobody will argue. The only things you do not control are Vespyra and the clock.

Who you can call, and for what

- Tower: what is on final, landing clearances.
- Approach: where every aircraft is, sequencing, go-arounds.
- Duty manager: runways, ground crews, how many the airport can take.
- Airline ops: fuel and crew limits, their fleet only.
- Fire and rescue: what they can cover.
- Titan Watch liaison: the clock, and how stale it is.
- FAA command centre: a national ground stop, slowly.
- Portland: divert capacity.

Each sees a piece. You see what they tell you, when they tell you. No screen of your own.

Your scope

In scope, you decide and order:

- Ground stop, yes or no, and when.
- Which aircraft land here, which divert where, which hold.
- Runway use, crew staging, when to stop saving everything and start choosing.

Out of scope:

- How a specialist carries out your order. They know their job.
- Where Vespyra is or goes. Titan Watch reports it; nobody controls it.

Anchors, fixed

- Every order you give takes minutes to reach the people who act on it, and every report reaches you late.
- No one has ever run this against a clock shorter than an hour.

Three phrases

"I do not know." - "Assume and label it." - "Ask the specialist, then tell me."

Next

Turn the page. The run sheet has every command and prompt for this scenario, in order.

How to proceed: Sea-Tac

Order of work, with every prompt filled in for this scenario. Full detail is in the shared instructions.

Lab 1: Discovery

- 1 `/clear`, then `/hve-core:dt-coach`. Paste:

Resume project seatac-ground-stop. Scope is done under `.copilot-tracking/dt/seatac-ground-stop/`; read it, including every interview there, do not redo it. I play the incident commander with full authority: the specialists in those interviews report to me and carry out my orders. I see only what they report. Interview me as that role, five questions, using what the specialists said. Then write `interview-commander-roleplay.md` under slug `seatac-ground-stop-commander`. Label everything `role-play`. Expect a software tool. Do not offer the canonical deck.

- 1 Answer five questions as the commander. Read your interview file once. Commit.

- 2 Same session, paste:

Project seatac-ground-stop. Read the shared scope under `.copilot-tracking/dt/seatac-ground-stop/`, every interview there, and my interview under `.copilot-tracking/dt/seatac-ground-stop-commander/`. Restate the outcome in light of the interviews, keeping the original beside it. Summarise the problem as if hearing it for the first time. List what you still need before a PRD. Then draft the PRD with non-functional requirements, close gaps with labelled assumptions, and state what it rests on. Then a technology-neutral diagram as Mermaid. Then an opinionated Azure overlay with the minimum services, rejected alternatives recorded, and a latency budget. Write everything under `.copilot-tracking/dt/seatac-ground-stop/`. Run without stopping. Label all interviews `role-play`.

- 1 PRD, diagram and overlay land under `dt/seatac-ground-stop/`. Commit.

Lab 2: Security and Responsible AI

Write three predicted security findings on paper first. Two terminals.

- 1 Terminal 1: `/clear`, then `/hve-core:security-planner`. Paste:

Use the security planner to plan the production security for this solution. PRD and architecture are under `.copilot-tracking/dt/seatac-ground-stop/`. Scoping answers are in `security-scoping-answers.md` in the same folder; treat every row as answered and labelled, do not ask them again. Threat-model the boundaries, map controls to OWASP, NIST and CIS, produce a backlog. Ask me only about severe or blocking issues. Run all phases without stopping. Prioritise by severity. Commit only, never push.

- 1 Terminal 2: `/clear`, then `/hve-core:rai-planner-agent`. Paste:

Run an RAI assessment on the single-save decision: the disaster-recovery AI in this app's Last Stand scene that recommends which one of two cities to save when only one can be saved. Today it outputs a single verdict with no confidence, no explanation and no audit trail, and a commander approves it under time pressure. Assess it against NIST AI RMF and treat the autonomous life-safety decision as the central risk. Use project name `single-save-decision`. Imagine the scoping answers, label them, run to completion unless blocking.

- 1 When the security planner finishes: "Plan summary by severity: counts per level, Critical items one line each, recommended next planner." Compare with your paper. Read the RAI go/no-go. Commit.

Lab 3: Research, Plan, Implement

- 1 Open your PRD under `dt/seatac-ground-stop/`. Pick one functional requirement the app could carry. One screen, one rule, one feed.
- 2 `/clear`, then `/hve-core:rpi-agent`. Ask: "What do you need from me to create an RPI plan?" Then paste, filled in:

Goal: implement <requirement, one sentence> from the PRD at `.copilot-tracking/dt/seatac-ground-stop/<prd file>`.
Scope: <one component or module>. Nothing else.
Acceptance criteria: <two or three testable lines>.
Constraints: no backend, no tracker, no cloud tenant; mock or defer and log the gap. No tracking paths in code or commits.
Validation: the existing test, lint and build scripts.
Context: the PRD and the Azure overlay in the same folder.
Mode: research, stop for my review; plan, stop for my approval.

- 1 Research stops: read it, fix the file if wrong, then `/hve-core:rpi-plan`.
- 2 Plan stops: amend one item in the plan file, tell the agent "plan amended in the file". Commit.
- 3 Homework: `/hve-core:rpi-implement`, then `/hve-core:rpi-review`.